

Full-Stack Web Development — Day 8 Study Material

MODULE
React & Next.js Foundations

SUBJECT
Next.js App Router, Routing & Server Components

AUTHOR
Penumuru Madhu Sudhan Reddy

Executive Summary: Practical, beginner-friendly guide to Next.js App Router. Covers file-based routing, server vs client components, dynamic segments ([id]), shared layout with responsive sidebar, loading skeletons, error boundaries with retry, and search parameter filtering.

1. Next.js vs Plain React

Feature	Plain React (Vite / CRA)	Next.js App Router
Routing	Virtual client library (react-router)	File-system hierarchy in app/ folder
Rendering	Client-side only (empty initial HTML)	Server Components by default (fast initial load)
Bundle Size	All JS shipped to browser	0 bytes sent for Server Components

2. App Router Conventions (Exercises 1 & 2)

Each folder in app/ corresponds to a URL path:

- app/page.tsx → / (Dashboard)
- app/projects/page.tsx → /projects (Project List)
- app/projects/[id]/page.tsx → /projects/1 (Dynamic Project Detail)
- app/projects/[id]/towers/[towerId]/page.tsx → /projects/1/towers/t1 (Nested Tower Detail)
- app/layout.tsx → Shared layout with persistent Sidebar and mobile bottom navigation.

3. Exercise 1 & 4: Search Parameters (?status=active)

```
// app/projects/page.tsx
export default async function ProjectsPage({ searchParams }: PageProps) {
  const { status } = await searchParams;
  const filtered = status ? projects.filter(p => p.status === status) : projects;
  return (
    <div>
      <h1>Projects List</h1>
      {filtered.map(p => <p key={p.id}>{p.name} - {p.status}</p>)}
    </div>
  );
}
```

Output (o/p):
URL: /projects?status=active
- Skyline Residency - active
- Metro Logistics Park - active

4. Exercise 3: Loading Skeletons & Error Boundaries

```
// app/loading.tsx
export default function Loading() {
  return <div className="skeleton">Loading site data ... </div>;
}

// app/error.tsx
'use client';
export default function ErrorPage({ error, reset }: { error: Error; reset: () => void }) {
  return (
    <div>
      <p>{error.message}</p>
      <button onClick={() => reset()}>Retry</button>
    </div>
  );
}
```